llnl_c1/lulesh · lulesh.cc · bare_llm_v4 · claude-opus-5-5 · FASTER

−86 / +217 lines of 1967 · 24 parallel region(s) added · 0 of 0 candidate(s) kept · RESTRUCTURING (174 code line(s) added, 77 removed) · restructuring level 0

BEFORE

```
109  void InitStressTermsForElems(Domain &domain,
110                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
111                               Index_t numElem)
112  {
113
114     for (Index_t i = 0 ; i < numElem ; ++i){
115        sigxx[i] = sigyy[i] = sigzz[i] =  - domain.p(i) - domain.q(i) ;
116     }
117  }
118
     …
298        fy[i] = -( stress_yy * B[1][i]  );
299        fz[i] = -( stress_zz * B[2][i] );
300     }
301  }
302
303  static inline
304  void IntegrateStressForElems( Domain &domain,
305                                Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
306                                Real_t *determ, Index_t numElem, Index_t numNode
307  {
308
```

AFTER

```
109  void InitStressTermsForElems(Domain &domain,
110                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
111                               Index_t numElem)
112  {
113
114  #pragma omp parallel for schedule(static) firstprivate(numElem) \
115          shared(domain, sigxx, sigyy, sigzz)
116     for (Index_t i = 0 ; i < numElem ; ++i){
117        sigxx[i] = sigyy[i] = sigzz[i] =  - domain.p(i) - domain.q(i) ;
118     }
119  }
120
     …
300        fy[i] = -( stress_yy * B[1][i]  );
301        fz[i] = -( stress_zz * B[2][i] );
302     }
303  }
304
305  /* Node -> element-corner map (corners listed in increasing element order),
306     built once and cached: the mesh connectivity never changes. */
307  static Index_t *s_nodeElemStart = NULL ;
308  static Index_t *s_nodeElemCorner = NULL ;
309  static Index_t  s_mapNumNode = -1 ;
310  static Index_t  s_mapNumElem = -1 ;
311  static const Index_t *s_mapNodelist = NULL ;
312
313  static void GetNodeElemMap(Domain &domain, Index_t numElem, Index_t numNode,
314                             Index_t **start, Index_t **corner)
315  {
316     const Index_t *nl = domain.nodelist(0) ;
317     if (s_nodeElemStart == NULL || s_mapNumNode != numNode ||
318         s_mapNumElem != numElem || s_mapNodelist != nl) {
319        Release(&s_nodeElemStart) ;
320        Release(&s_nodeElemCorner) ;
321        s_nodeElemStart  = Allocate<Index_t>(numNode + 1) ;
322        s_nodeElemCorner = Allocate<Index_t>(8 * numElem + 1) ;
323        Index_t *fill = Allocate<Index_t>(numNode + 1) ;
324        for (Index_t n = 0 ; n <= numNode ; ++n) {
325           s_nodeElemStart[n] = 0 ;
326        }
327        for (Index_t c = 0 ; c < 8 * numElem ; ++c) {
328           s_nodeElemStart[nl[c] + 1]++ ;
329        }
330        for (Index_t n = 0 ; n < numNode ; ++n) {
331           s_nodeElemStart[n + 1] += s_nodeElemStart[n] ;
332        }
333        for (Index_t n = 0 ; n < numNode ; ++n) {
334           fill[n] = s_nodeElemStart[n] ;
335        }
336        for (Index_t c = 0 ; c < 8 * numElem ; ++c) {
337           s_nodeElemCorner[fill[nl[c]]++] = c ;
338        }
339        Release(&fill) ;
340        s_mapNumNode = numNode ;
341        s_mapNumElem = numElem ;
342        s_mapNodelist = nl ;
343     }
344     *start = s_nodeElemStart ;
345     *corner = s_nodeElemCorner ;
346  }
347
348  /* Adds per-corner forces to the nodal forces, in element order per node
349     (same floating-point summation order as the original scatter). */
350  static void GatherCornerForces(Domain &domain, Index_t numNode,
351                                 Index_t *nodeElemStart, Index_t *nodeElemCorner
352                                 Real_t *fx_elem, Real_t *fy_elem, Real_t *fz_el
353  {
354  #pragma omp parallel for schedule(static) firstprivate(numNode) \
355          shared(domain, nodeElemStart, nodeElemCorner, fx_elem, fy_elem, fz_ele
356     for (Index_t n = 0 ; n < numNode ; ++n) {
357        Real_t fx = domain.fx(n) ;
358        Real_t fy = domain.fy(n) ;
359        Real_t fz = domain.fz(n) ;
360        Index_t cEnd = nodeElemStart[n + 1] ;
361        for (Index_t c = nodeElemStart[n] ; c < cEnd ; ++c) {
362           Index_t ci = nodeElemCorner[c] ;
363           fx += fx_elem[ci] ;
364           fy += fy_elem[ci] ;
365           fz += fz_elem[ci] ;
366        }
367        domain.fx(n) = fx ;
368        domain.fy(n) = fy ;
369        domain.fz(n) = fz ;
370     }
371  }
372
373  static inline
374  void IntegrateStressForElems( Domain &domain,
375                                Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
376                                Real_t *determ, Index_t numElem, Index_t numNode
377  {
378    Index_t *nodeElemStart ;
379    Index_t *nodeElemCorner ;
380    GetNodeElemMap(domain, numElem, numNode, &nodeElemStart, &nodeElemCorner) ;
381
```

file

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is